



Binding User Guide

MICROCHIP/PIC32CM GC00

Cortex-M0+/MPLAB X IDE

Version 5.3.0

Published September 1, 2025

Table of Contents

About this guide	iii
Conventions	iv
Feedback	iv
Chapter 1: Distribution Overview	5
Chapter 2: Binding Information	7
Memory Requirements	7
Startup Code	8
PendSV Context Switching	8
PX5 RTOS Periodic Timer Interrupt	9
Application Interrupt Service Routines (ISRs)	9
Context Stack Frame	9
Exception Stack Frame	11
Chapter 3: Examples	12
Basic Example	12
Message Send and Receive Example	15
Pthread Condition Variable Signal and Wait Example	15
Pthread Create Example	15
Pthread Event Flags Set and Wait Example	16
Pthread Fastqueue Send and Receive Example	16
Pthread Memory Allocate and Free Example	16
Pthread Mutex Lock and Unlock Example	16
Pthread Partition Allocate and Free Example	16
Pthread Ticktimer Create Example	17
Semaphore Post and Get Example	17



About this guide

This guide provides embedded developers with the information necessary to successfully utilize the PX5 Real-Time Operating System (RTOS) on the Cortex-M0 architecture using the GCC development tools. This guide is organized into a series of chapters as follows:

Chapter 1: Distribution Overview. This chapter describes the distribution overview, including where to find the documentation, source code, and examples for the *MICROCHIP/PIC32CM GC00*.

Chapter 2: Binding Information. This chapter describes the specific *MICROCHIP/PIC32CM GC00* support and requirements of the PX5 RTOS.

Chapter 3: Examples. This chapter provides an overview of the examples provided for PX5 RTOS running on the *MICROCHIP/PIC32CM GC00*.

Conventions

Various conventions are used in this guide. C source code function prototypes and examples are written in `courier` font. Supplemental documentation, API, and parameter names are italicized when discussed. Several symbols are used to highlight important features or topics, as follows:

Symbol	Meaning
	This is a general information symbol.
	The caution symbol indicates that the user should know important usage scenarios associated with a specific topic or API.
	The danger symbol indicates that the user should know the serious consequences of certain scenarios associated with a specific topic or API.
	This symbol indicates cases where the associated API scenario does not result in preemption.
	This symbol indicates the case where the associated API scenario results in preemption.
	This symbol indicates the case where the associated API scenario results in suspension.

Feedback

We greatly appreciate all feedback. Please send feedback to support@px5rtos.com with "PX5 RTOS Binding User Guide Feedback" in the subject line.

Chapter 1: Distribution Overview

This chapter describes the PX5 RTOS distribution for the *MICROCHIP PIC32CM GC00 (Cortex-M0)* architecture using the *MPLAB X IDE (GCC)* development tool, including where to find the documentation, source code, and examples. The distribution is contained in a single compressed folder (zip file) and, when extracted, contains the following folders:

```
px5_rtos_release
  documentation
  examples
    basic_example
    message_send_receive_example
    pthread_condition_var_signal_wait_example
    pthread_create_example
    pthread_event_flags_set_wait_example
    pthread_fastqueue_send_receive_example
    pthread_memory_allocate_free_example
    pthread_mutex_lock_unlock_example
    pthread_partition_allocate_free_example
    pthread_ticktimer_create_example
    semaphore_post_get_example
  px5_rtos
    bits
    sys
```

The *documentation* folder contains all the PX5 RTOS *MICROCHIP/PIC32CM GC00* support documentation, including this document and the generic *PX5_RTOS_User_Guide.pdf*.

The *examples* folder contains all the PX5 RTOS *MICROCHIP/PIC32CM GC00* examples provided with the distribution. The *basic_example* is always provided. If present, additional examples will be in separate folders within the *examples* folder. The *basic_example* folder is representative of all other examples and has the following format:

```
basic_example
  firmware
    basic_example.X
    src
  hex
```

The *firmware* folder contains all the project files, where the *basic_example.X* folder is recognized by *MPLAB X IDE* as the project. The *src* folder contains all the source files of the project, and the *hex* folder contains the final ELF image of the project. All other example projects share a similar structure.

The *px5_rtos* folder contains all of the PX5 RTOS source code for execution on the Cortex-M0 architecture using the *MPLAB X IDE (GCC)* development tools. Note that the source folder has additional sub-folders that effectively mimic the directory structure of most POSIX pthread distributions.



The only MICROCHIP/PIC32CM GC00-specific source files are px5_binding.S and px5_binding.h. All the other source files in the distribution are generic ANSI C files.

Chapter 2: Binding Information

This chapter describes the PX5 RTOS support for Cortex-M0 using the GCC development tools, including all expectations of the PX5 RTOS. The following subsections provide detailed information.

Memory Requirements

The PX5 RTOS requires memory for its instructions and global data. For minimal usage, the instruction area memory requirement is on the order of 1KB. If the application uses every service PX5 RTOS provides, the instruction area memory requirement is on the order of 25KB. The actual amount of memory automatically scales based on precisely what the application utilizes.

RAM is required for the PX5 RTOS global data (encapsulated in the *px5_globals* variable). For most Cortex-M0 devices, the amount of RAM required is under 1KB. PX5 RTOS also requires both a PSP and MSP stack. These are typically defined in the linker control file and range from 1KB to 4KB. Please see the linker control file for the exact size.

During runtime, the PX5 RTOS provides services for the dynamic creation of various system objects, including *threads*, *mutexes*, *condition variables*, *event flags*, *fastqueues*, *memory pools*, *partition pools*, *message queues*, *semaphores*, and *ticktimers*. Each of these system objects requires RAM. The following table shows the RAM requirements for each system object:

Object	Control Block	Additional Memory
<i>Thread</i>	284 bytes	Thread stack (default size 4KB, user-specified)
<i>Condition Var</i>	52 bytes	None
<i>Event Flags</i>	48 bytes	None
<i>Fastqueue</i>	84 bytes	User-specified queue memory
<i>Memory Pool</i>	156 bytes	User-specified memory for the pool

<i>Message Queue</i>	88 bytes	User-specified queue memory
<i>Mutex</i>	80 bytes	None
<i>Partition Pool</i>	64 bytes	User-specified memory for the partition pool
<i>Semaphore</i>	48 bytes	None
<i>Ticktimer</i>	84 bytes	None

Startup Code

The PX5 RTOS requires that interrupts be disabled when *main* is entered. When *px5_pthread_start* is called, PX5 RTOS initializes the *PSP* stack pointer to the current stack pointer, which must reside in the stack area set up during *PIC32CM GC00* initialization (between the symbols *__StackLimit* and *__StackTop*). For interrupt processing, PX5 RTOS also sets up the *MSP* stack. The size and location of the *MSP* stack are defined in *px5_binding.S*.

The *main* function is then elevated to the first thread in the system.



Note that MPLAB X IDE defines the `_stack` symbol by default at the end of the RAM memory area (typically the `_ram_end_` symbol defined by the linker control file). This is the symbol the startup code uses to setup the initial SP. Given this, the PX5 RTOS support for the PIC32CM GC00 adds lines at the bottom of the linker control file to map the lowest stack address to `__StackLimit` and the highest stack address to `__StackTop` (assuming a minimal 1024 stack area).

PendSV Context Switching

The PX5 RTOS uses the *PendSV* interrupt to perform context switching. The *px5_binding.s* file defines the *PendSV_Handler*, which is automatically installed in the Cortex-M0 vector table (the default GCC handler is defined as a weak symbol and is, therefore, overridden by the PX5 RTOS definition).



The PendSV interrupt is set to the lowest priority in the `px5_binding_initialize` function in `px5_binding.s`. All other interrupts used in the application should be given higher priority than PendSV.

PX5 RTOS Periodic Timer Interrupt

The PX5 RTOS needs a single, periodic timer interrupt to drive all of its time-related services. On the Cortex-M0, this is typically the SysTick timer and the handler is defined at the label *SysTick_Handler*.

As for the timer interrupt period, most applications use a 1ms interrupt interval. However, the application can use any timer interrupt frequency, provided that the PX5 RTOS is built with the correct *PX5_TICKS_PER_SECOND* (the default is 1,000, representing a 1ms timer interrupt interval).

From the timer interrupt handler, the PX5 RTOS routine *px5_rtos_interrupt_process* must be called on each timer interrupt.



As mentioned previously, the timer interrupt must have a higher priority than the PendSV interrupt. The application determines exactly how high the priority should be.

Application Interrupt Service Routines (ISRs)

Application Interrupt Service Routines (ISRs) are effectively just C functions on the Cortex-M0 using the GCC development tools.

Context Stack Frame

The PX5 RTOS stores the context of non-running threads on their stack. The *px5_thread_control* typedef *current_stack_pointer* member points to the context of the non-executing thread in this format:

Offset	Stack Contents
0	R4
4	R5
8	R6
12	R7
16	R8

20	R9
24	R10
28	R11
32	Exception LR
36	R0
40	R1
44	R2
48	R3
52	R12
56	R14 (LR)
60	PC (Point of Interrupt)
64	xPSR
68	Reserved
72	Reserved
76	Reserved
80	Reserved
84	Reserved

Values in **red** (above) represent the registers/information placed on the stack via the Cortex-M0 exception processing.

Exception Stack Frame

When an exception occurs, the Cortex-M0 hardware pushes various information on the MSP stack and then jumps to the specific exception handler. The format of the information on the stack is shown below:

MSP Offset Stack Contents

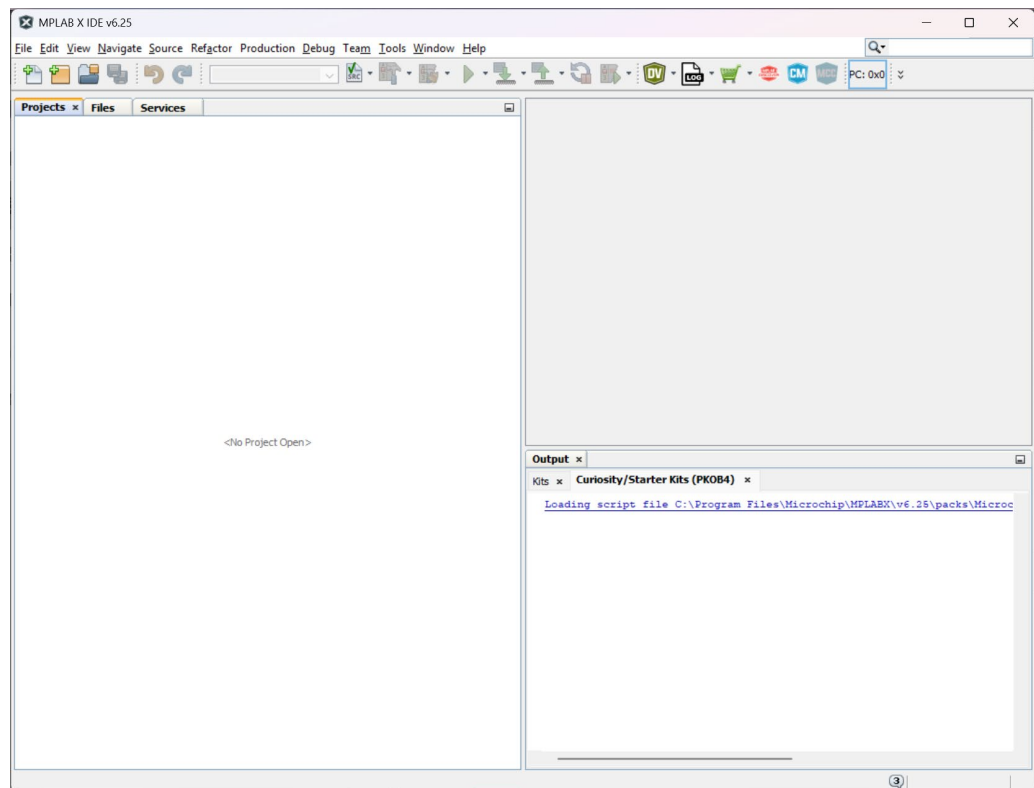
0	R0
4	R1
8	R2
12	R3
16	R12
20	R14 (LR)
24	PC (Exception Address)
28	xPSR

Chapter 3: Examples

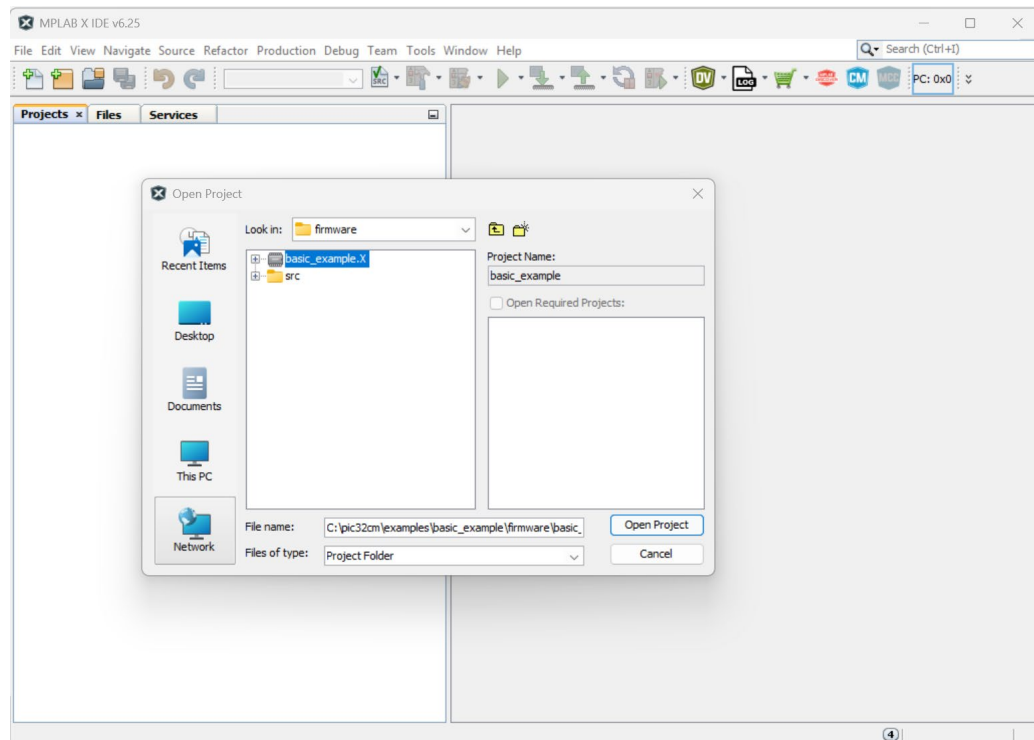
The PX5 RTOS *PIC32CM GC00* examples are found in the *example* folder of the distribution. Each sub-section below defines each example within the distribution.

Basic Example

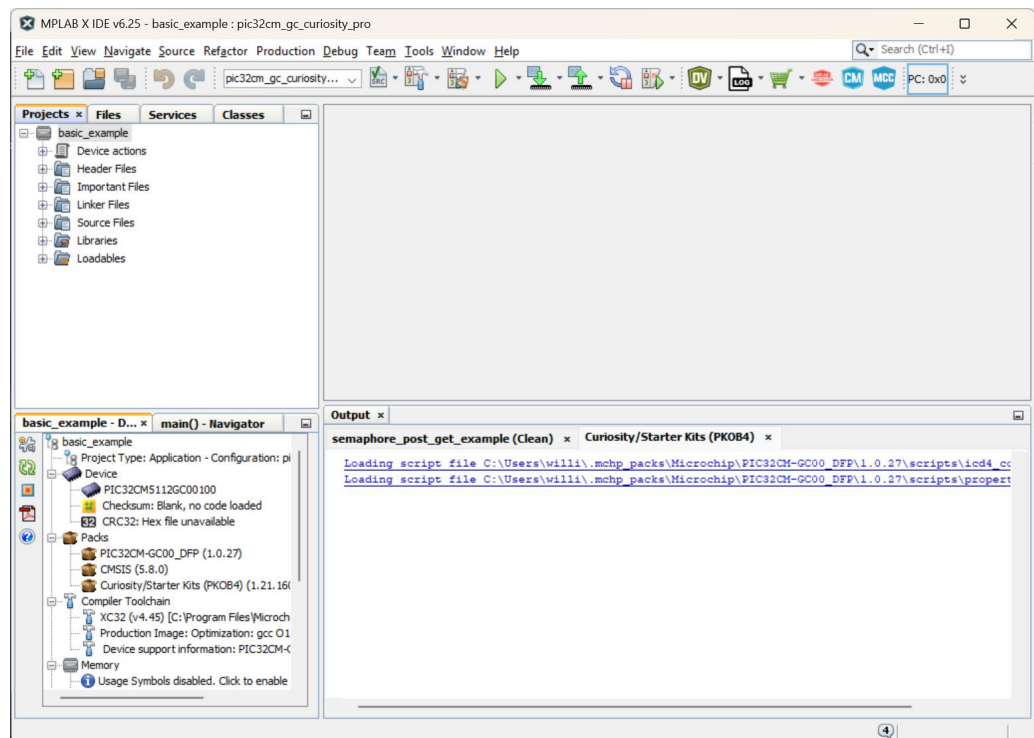
The basic example demonstrates how to build and start the PX5 RTOS, and then observe execution within a while loop of the main thread. To build the *basic_example*, invoke *MPLAB X IDE* as shown below:



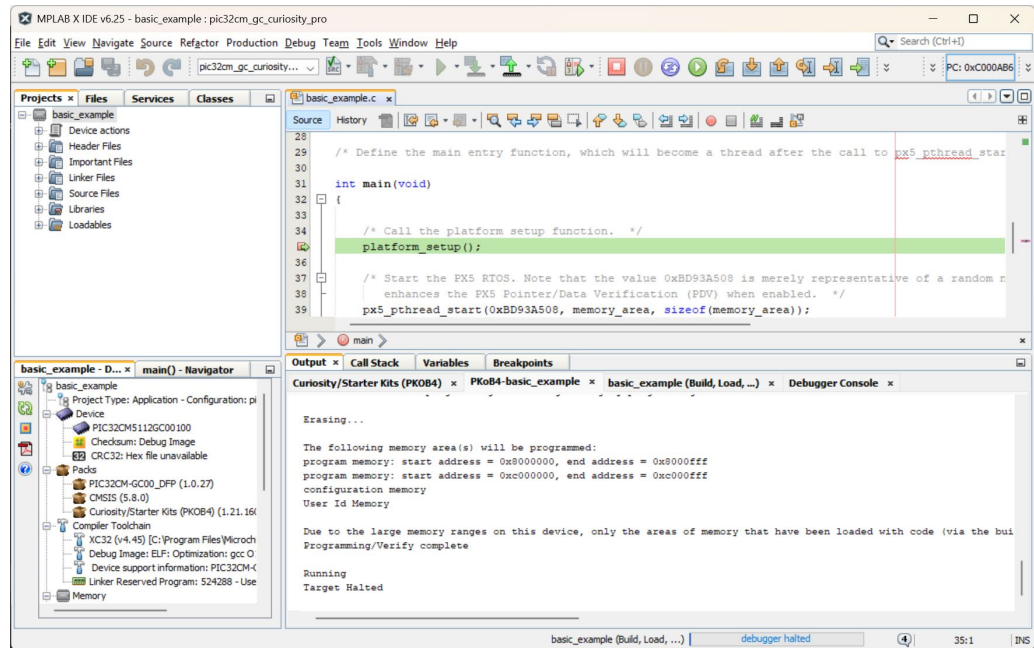
Next, open the *basic_example* project by navigating to the project folder in the distribution, as shown below:



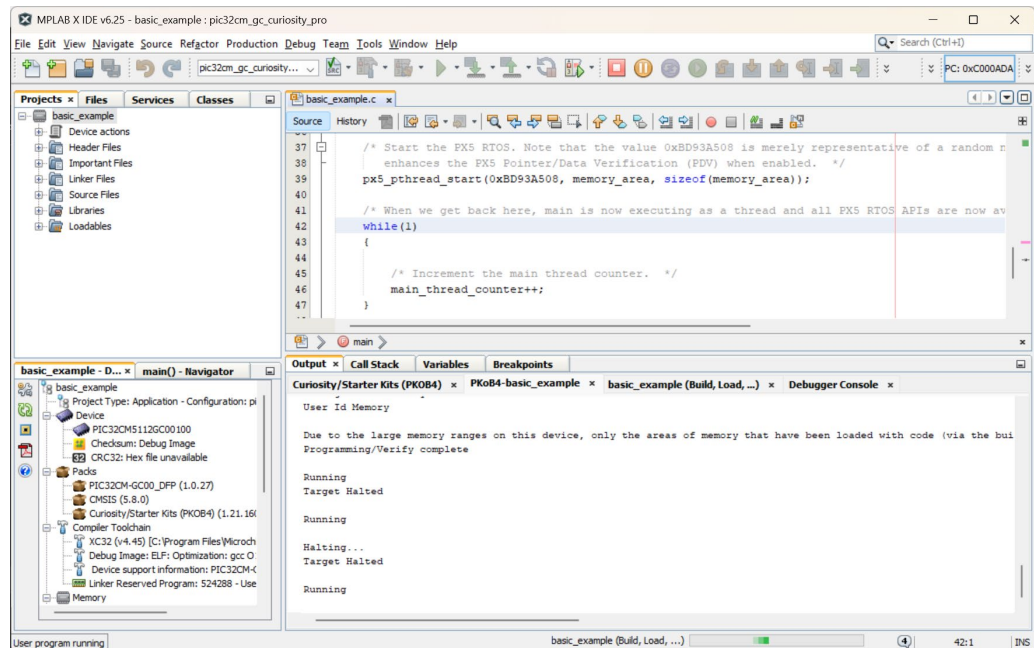
You will then observe the *basic_example* in under Projects in the IDE as shown below:



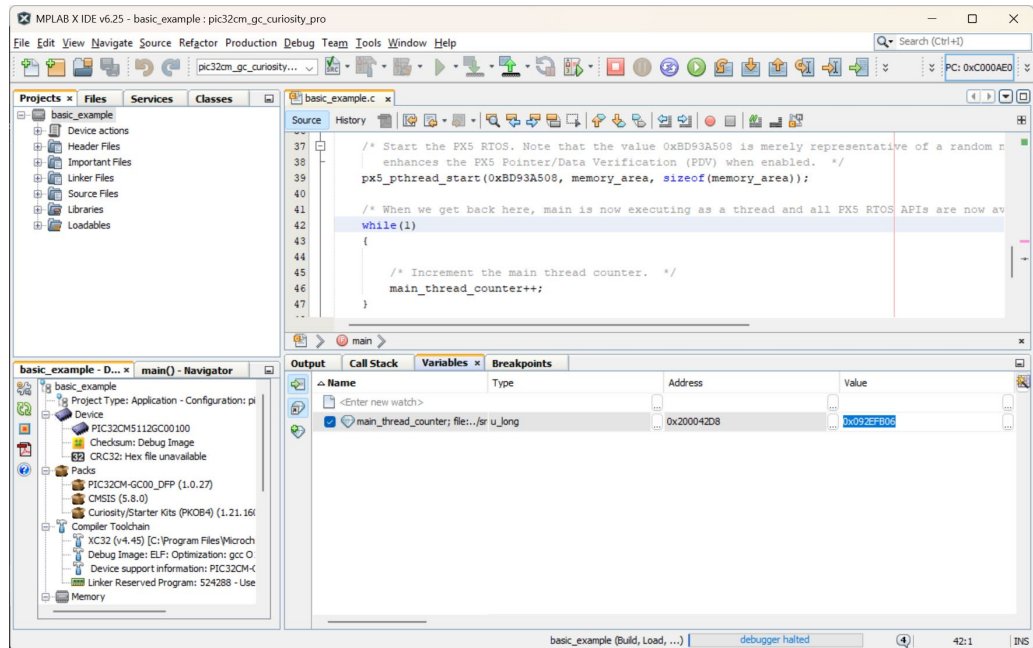
Select “*Debug -> Debug Project*” from the menu to build and download the *basic_example* project to the *PIC32CM GC00* evaluation board, as shown below:



You are now ready to execute the example. Execute the “*Debug -> Continue*” menu option, or the *Continue* button, or *F5* to start execution. You will then see the following display:



To halt execution, press “*Debug -> Pause*” or the *Pause* button, which is shown below with the *Variables* tab selected, where *main_thread_counter* is being watched:



All of the other examples in the distribution can be executed in a similar fashion.

Message Send and Receive Example

This example shows how to use POSIX message queues to send and receive messages. The *main_thread_counter* and *child_thread_counter* represent the number of times the main thread has sent a message to the child thread.

Pthread Condition Variable Signal and Wait Example

This example shows how to use the POSIX pthread condition variable construct to communicate between the main and child threads. The *main_thread_counter* and *child_thread_counter* represent the number of times the main thread has signaled the child thread via a condition variable.

Pthread Create Example

This example shows how to use the POSIX pthread create construct to create a default-priority child thread (same as the main thread) and a higher-priority child thread. The *main_thread_counter*, *child_thread_counter*, and *prioritychild_thread_counter* represent the number of times each thread has

executed its processing loop. Notably, the main thread and the child thread execute at the same priority so they can share the processor via the *sched_yield* API, while the higher-priority thread that preempts their execution sleeps for one timer tick.

Pthread Event Flags Set and Wait Example

This example shows how to use the POSIX pthread event flags construct (a PX5 RTOS POSIX extension) to synchronize execution between the main and child threads. The *main_thread_counter* and *child_thread_counter* represent the number of times the main thread has set events that the child thread waits for.

Pthread Fastqueue Send and Receive Example

This example shows how to use the POSIX pthread fastqueue construct (a PX5 RTOS POSIX extension) to communicate between the main thread and the child thread swiftly. The *main_thread_counter* and *child_thread_counter* represent the number of times the main thread has sent a message to the child thread.

Pthread Memory Allocate and Free Example

This example shows how to use the POSIX pthread variable-length memory pool construct (a PX5 RTOS POSIX extension) to allocate and release memory. The *main_thread_counter* and *child_thread_counter* represent the number of times the main thread and child thread have allocated and released memory.

Pthread Mutex Lock and Unlock Example

This example shows how to use the POSIX pthread mutex construct for mutual exclusion. The *main_thread_counter* and *child_thread_counter* represent the number of times the main thread and child thread have obtained and released the mutex they compete for.

Pthread Partition Allocate and Free Example

This example shows how to use the POSIX pthread fixed-length partition pool construct (a PX5 RTOS POSIX extension) to allocate and release fixed-sized memory. The *main_thread_counter* and *child_thread_counter* represent the number of times the main thread and child thread have allocated and released partition memory.

Pthread Ticktimer Create Example

This example shows how to use the POSIX pthread ticktimer construct (a PX5 RTOS POSIX extension) to execute user-supplied functions on a one-shot or periodic basis based on the timer interrupt (tick). The *ticktimer_counter* represents the number of timer ticks that have occurred since the ticktimer was set to expire every timer tick.

Semaphore Post and Get Example

This example shows how to use POSIX semaphores in a consumer-producer fashion. The *main_thread_counter* and *consumer_child_thread_counter* represent the number of times the main thread has sent a semaphore to the child thread for processing.



Enhance • Simplify • Unite

11440 West Bernardo Court • Suite 300
San Diego, CA 92127, USA

Phone: +1 (858) 753-1715
Website: px5rtos.com

© PX5 • All Rights Reserved